End-to-End Neural Network for Direct Portfolio Weight Optimization

Muhammad Zaid Maqsood
Department of Computer & Information Sciences
Fordham University
New York City, USA
mm456@fordham.edu

***Abstract*— Portfolio optimization is difficult because investors must increase returns while controlling risk and avoiding unnecessary allocation changes. Traditional portfolio construction usually estimates future returns first and then uses those estimates to calculate portfolio weights. However, inaccurate return estimates can affect the final allocation and may not capture nonlinear market behavior. This study builds an end-to-end machine learning framework that learns portfolio weights directly from historical market signals. Using ten years of daily data for ten diversified exchange-traded funds, I created features including daily returns, lagged returns, rolling volatility, moving average ratios, and momentum indicators. I then compared a Multi-Layer Perceptron and Long Short-Term Memory model against a Mean-Variance Optimization benchmark. The models were evaluated using cumulative return, annual return, annual volatility, Sharpe ratio, and maximum drawdown. The Multi-Layer Perceptron achieved the highest cumulative return of 87.76% and the strongest Sharpe ratio of 2.1061. The Long Short-Term Memory model also improved annual return and Sharpe ratio compared with the benchmark but had a larger maximum drawdown. Overall, the results show that end-to-end machine learning can improve portfolio allocation, but higher returns may come with greater risk exposure.**

Keywords—Portfolio Optimization, Machine Learning, Neural Networks, LSTM, MLP, Mean-Variance Optimization, Sharpe Ratio, Financial Time Series

I. Introduction

Financial portfolio construction requires investors to decide how capital should be divided across multiple assets as market conditions change. The objective is not only to increase returns, but also to control downside risk, avoid unnecessary allocation changes, and keep the portfolio stable across different market environments. Traditionally, this problem is handled through a two-step process where expected returns are first estimated and then used in an optimization model to determine portfolio weights. Mean-Variance Optimization, originally introduced by Markowitz [1], remains one of the most established methods because it provides a structured way to balance expected return and portfolio variance. However, this approach depends heavily on return estimates, which are difficult to predict in financial markets due to volatility, noise, and changing market conditions.

This dependence creates a practical issue in portfolio construction. If expected returns are estimated incorrectly, the final portfolio weights can also become unreliable. In other words, errors from the prediction stage can pass directly into the optimization stage. This issue becomes more important when asset relationships change over time or when market behavior follows patterns that are not easily captured through linear assumptions. Recent work in empirical asset pricing also shows that machine learning models can capture complex return patterns that traditional linear methods may miss [2]. As a result, two-step portfolio optimization may not always produce stable or effective allocation decisions in real market settings.

To address this limitation, this study explores an end-to-end machine learning approach that learns portfolio allocation weights directly from historical market signals instead of predicting returns first and optimizing weights later. This approach is also aligned with recent deep learning research in finance, where neural networks have been used to learn portfolio representations and improve allocation decisions [3], [4]. The study uses a diversified exchange-traded fund portfolio and evaluates whether neural network-based allocation models can improve performance compared with a Mean-Variance Optimization benchmark. The main contribution is to compare traditional optimization with direct machine learning-based allocation under practical investment criteria, including risk-adjusted performance and portfolio stability.

Therefore, the main research question is whether end-to-end machine learning models can directly learn portfolio allocation weights and outperform a Mean-Variance Optimization benchmark in risk-adjusted performance and portfolio stability. By answering this question, the study evaluates whether direct learning can provide a more flexible and data-driven approach to portfolio construction while still considering risk exposure and allocation movement.

II. Data and Methodology

A. Data

This study uses historical financial market data collected from Yahoo Finance through the open-source yfinance Python library [5]. The dataset includes daily adjusted closing prices for ten diversified exchange-traded funds from January 2015 to December 2025. An exchange-traded fund, or ETF, trades like an individual stock but gives exposure to a wider group of assets, such as equities, bonds, commodities, or real estate securities. ETFs were selected because they provide a simple way to build a diversified portfolio across different areas of the financial market. The selected ETFs are SPY, QQQ, IWM, EFA, EEM, TLT, LQD, GLD, VNQ, and XLE, covering United States equities, international markets, bonds, gold, real estate, and energy. This mix allowed the study to evaluate how different

models allocate capital across assets with different risk and return behavior.

Ticker	EEM	EFA	GLD	IWM	LQD	QQQ	SPY	TLT	VNQ	XLE
Date										
2015-01-02	30.355043	43.780632	114.080002	102.786987	80.710709	94.665100	170.125031	93.367317	52.549866	25.548061
2015-01-05	29.814816	42.747181	115.800003	101.412796	81.040695	93.276451	167.052597	94.833939	52.837479	24.491192
2015-01-06	29.689545	42.262981	117.120003	99.658333	81.370773	92.025764	165.479080	96.542603	53.361568	24.131397
2015-01-07	30.331570	42.732727	116.430000	100.885597	81.478508	93.212067	167.541199	96.351944	54.179668	24.182796
2015-01-08	30.848314	43.310871	115.940002	102.596817	81.215843	94.996124	170.514252	95.075966	54.384186	24.725702

Fig. 1. Dataframe Head

After collecting the adjusted price data, daily returns were calculated and used for feature engineering. Raw prices alone do not fully explain market movement, risk behavior, or short-term changes across assets. Therefore, additional market-based features were created, including lagged returns, rolling volatility, moving average ratios, and momentum indicators. Lagged returns captured recent price movement, while rolling volatility measured changing risk conditions over time. Moving average ratios showed how current prices compared with recent trends, and momentum indicators captured the strength and direction of movement across different time windows. This helped the models learn from both short-term movements and broader market trends instead of relying only on raw price data.

B. Methodology

The dataset was split chronologically into training, validation, and testing periods to preserve the temporal structure of financial time series data and prevent data leakage. Unlike random splitting, chronological splitting ensures that the models are trained only on past observations and evaluated on future unseen data. This setup better reflects a real investment environment where portfolio decisions must be made using information available up to that point. The final split used 70% of the data for training, 15% for validation, and 15% for testing.

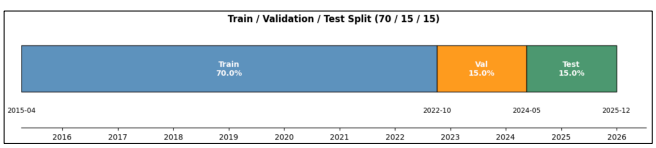

Fig. 2. Data Split Distribution

Before training the machine learning models, the engineered features were standardized using the training data and then transformed across the validation and test sets. This step ensured that the models learned from features placed on a consistent scale without allowing information from the validation or test periods to influence the training process.

A traditional Mean-Variance Optimization benchmark was developed to compare the machine learning models against a standard portfolio allocation approach. This benchmark used historical mean returns and the covariance matrix from the training period to estimate portfolio weights by maximizing the Sharpe ratio under allocation constraints. A maximum weight limit was applied to each asset to avoid excessive concentration and maintain diversification. The optimized weights were then applied to the test period to evaluate out-of-sample performance.

To evaluate machine learning-based portfolio allocation, two neural network models were developed. The Multi-Layer Perceptron model was used to learn nonlinear relationships from flattened engineered features, while the Long Short-Term Memory model was used to capture sequential market patterns from rolling time windows. Both models generated long-only portfolio weights through a "softmax" output layer, ensuring that the final allocation summed to one across all assets.

The machine learning models were trained using a custom portfolio-level objective function instead of a standard prediction loss. The loss function combined negative Sharpe ratio, turnover penalty, and drawdown penalty. The Sharpe ratio was used because it provides a standard way to evaluate return earned per unit of risk [6]. This allowed the models to optimize for Sharpe-based performance while limiting unnecessary allocation changes and reducing portfolio drawdown from prior peak values. Final performance was evaluated using cumulative return, annualized return, annualized volatility, Sharpe ratio, maximum drawdown, and portfolio turnover.

$$Loss = -\,Sharpe\ Ratio + \lambda_1 x\ Turnover + \lambda_2 x\ Drawdown \qquad (1)$$

III. Model Construction and Experiment Design

After defining the data pipeline and overall methodology, the next stage focused on constructing each portfolio model and testing different parameter settings. The purpose of this section is to describe how the benchmark, MLP, and LSTM models were configured and how the final model versions were selected. Since the goal was direct portfolio allocation, all neural network models produced portfolio weights across the ten ETFs rather than separate return predictions.

A. Benchmark Model: Mean-Variance Optimization (MVO)

The first model was the Mean-Variance Optimization portfolio, which was used as the traditional benchmark. This model was constructed using only the training period to estimate the expected return vector and covariance matrix. The objective was to maximize the Sharpe ratio by minimizing the negative Sharpe ratio. This design allowed the benchmark to identify the portfolio allocation that provided the best return per unit of risk based on historical training data.

To make the benchmark more practical, allocation constraints were added during optimization. Each ETF was restricted to a maximum weight of 20%, and the full portfolio was required to sum to 100%. This prevented the optimizer from concentrating too heavily in only one or two assets, which can happen when expected returns are estimated from noisy financial data. The optimization was solved using Sequential Least Squares Programming, with equal initial weights assigned to all assets before the optimization process started. The final optimized benchmark allocation placed 20% each in GLD, QQQ, SPY, and TLT, 15.51% in LQD, and 4.49% in XLE, while the remaining ETFs received zero allocation.

The MVO portfolio was static after construction. Once the weights were estimated from the training period, the same allocation was applied throughout the test period. This made the benchmark useful for comparison because it represented a conventional optimization approach that does not adapt its weights dynamically during testing. Therefore, any improvement from the neural network models could be interpreted as the value added by direct learning from market

features rather than relying only on historical mean and covariance estimates.

B. Multi-Layer Perceptron Model (MLP)

The first neural network model was a Multi-Layer Perceptron. This model was designed to learn nonlinear relationships across the engineered financial features without explicitly modeling time sequence behavior. For each observation, the input consisted of a 21-day rolling window with 110 engineered features. These features were flattened into a single input vector of 2,310 values before being passed into the neural network. The output layer contained ten neurons, one for each ETF, and a "softmax" activation function was applied so that all predicted weights summed to one. This made the output directly usable as a portfolio allocation.

The MLP architecture used fully connected linear layers followed by ReLU activation and dropout regularization. ReLU was used to introduce nonlinearity, while dropout was used to reduce overfitting by randomly deactivating some neurons during training. The final layer transformed the hidden representation into ten asset weights. Unlike a standard prediction model, the MLP was not trained to predict next-day returns directly. Instead, it learned weights that performed well under the portfolio-level loss function.

```
MLP(
  (net): Sequential(
    (0): Linear(in_features=2310, out_features=64, bias=True)
    (1): ReLU()
    (2): Dropout(p=0.3, inplace=False)
    (3): Linear(in_features=64, out_features=10, bias=True)
    (4): Softmax(dim=-1)
  )
)

Input dim : 2310  (21 days × 110 features)
Output dim  : 10 assets
Parameters  : 148,554
```

Fig. 3. Multi-Layer Perceptron Architecture

Several MLP configurations were tested to evaluate the effect of model capacity and regularization. The baseline model used one hidden layer with 64 hidden units, a dropout rate of 0.2, a learning rate of 0.001, and 100 training epochs. Additional experiments increased the hidden size to 128 or 256, used two hidden layers, changed dropout between 0.3 and 0.5, and extended the training period to 150 or 200 epochs. Across all MLP experiments, the turnover and drawdown penalty weights were both set to 0.5, meaning the model was encouraged to improve return per unit of risk while also controlling allocation changes and downside risk. The best MLP configuration was Experiment 4. This version used 128 hidden units, two hidden layers, a dropout rate of 0.4, a learning rate of 0.0003, and 200 epochs. It achieved the lowest validation loss among the MLP models, with a best validation loss of -1.7935. Since the validation loss was based on the custom portfolio loss function, this result indicated that Experiment 4 provided the strongest balance of Sharpe improvement, turnover control, and drawdown penalty among the MLP configurations.

```
MLP Experiment Summary

Label          Best Val  Final Train  Final Val
---------------------------------------------
Baseline        -1.3535      -5.5737    -0.5856
Exp 1           -1.7017      -5.4955    -1.0049
Exp 2           -1.6853      -5.4703    -1.4716
Exp 3           -1.3362      -5.7506    -0.4382
Exp 4           -1.7935      -6.0146    -1.1329

Best MLP : Exp 4 | Best Val Loss : -1.7935
```

Fig. 4. Multi-Layer Perceptron Validation Results

C. Long Short-Term Memory Model (LSTM)

The second neural network model was a Long Short-Term Memory network. The LSTM architecture was originally introduced to address the difficulty of learning long-term dependencies in sequential data [7]. This model was included because financial data is sequential, and recent market movements may influence future allocation decisions. Unlike the MLP, which flattened the rolling window into one vector, the LSTM preserved the time dimension of the input. This allowed the model to process market signals across the 21-day window step by step and learn temporal dependencies in the data.

For the LSTM model, the input was reduced to features more directly related to sequential movement. These included daily returns, lagged returns, and momentum features. This produced 60 features per timestep across a 21-day sequence. The model then passed this sequence through an LSTM layer, followed by layer normalization, dropout, an intermediate fully connected layer, ReLU activation, and a final fully connected layer. A "softmax" activation was again applied at the output so that the model generated valid portfolio weights across the ten ETFs.

The LSTM architecture also included weight initialization steps to improve training stability. Orthogonal initialization was used for recurrent weights, Xavier initialization was used for input weights, and the bias terms were initialized to zero. This was important because recurrent neural networks can be sensitive to initialization, especially when trained on noisy financial data. The base LSTM model used 60 input features per timestep, a 21-day sequence length, ten output assets, and 238,538 trainable parameters in the displayed configuration.

```
LSTM(
  (lstm): LSTM(60, 128, num_layers=2, batch_first=True, dropout=0.3)
  (norm): LayerNorm((128,), eps=1e-05, elementwise_affine=True)
  (dropout): Dropout(p=0.3, inplace=False)
  (fc1): Linear(in_features=128, out_features=64, bias=True)
  (relu): ReLU()
  (fc2): Linear(in_features=64, out_features=10, bias=True)
  (softmax): Softmax(dim=-1)
)

Input dim   : 60 features per timestep
Sequence    : 21 days
Output dim  : 10 assets
Parameters  : 238,538
```

Fig. 5. Long Short-Term Memory Architecture

Five LSTM configurations were tested. The baseline model used 64 hidden units, one LSTM layer, no dropout, a learning rate of 0.001, and 100 epochs. The later experiments increased the hidden size to 128 or 256, used two LSTM layers, introduced dropout between 0.2 and 0.25, reduced the learning rate, and increased the number of epochs up to 250. The LSTM experiments also tested different turnover and drawdown

penalty combinations, with turnover penalties between 0.2 and 0.3 and drawdown penalties between 0.2 and 0.7. Gradient clipping was used for the LSTM models to improve training stability. The best LSTM configuration was Experiment 3. This model used 256 hidden units, two LSTM layers, a dropout rate of 0.2, a learning rate of 0.0003, and 250 epochs. It achieved the lowest validation loss among the LSTM configurations, with a best validation loss of -1.3259. Although this validation loss was not as low as the best MLP validation loss, it still represented the strongest sequence-based model in the experiment design.

LSTM Experiment Summary

Label	Best Val	Final Train	Final Val
Baseline	-1.3128	-1.9803	-0.8030
Exp 1	-1.2131	-2.2664	-0.7349
Exp 2	-1.2270	-1.9634	-1.0798
Exp 3	-1.3259	-2.0631	-0.8863
Exp 4	-1.1745	-2.4508	-0.8498

Best LSTM : Exp 3 | Best Val Loss : -1.3259

Fig. 6. Long Short-Term Memory Validation Results

D. Training and Model Selection

After constructing the MLP and LSTM architectures, both neural network models were trained using the same general training pipeline. The models were optimized using the Adam optimizer with weight decay to improve training stability and reduce overfitting. The training loop used mini-batches of size 64, where portfolio weights were generated for each batch and evaluated using the custom portfolio-level loss function. For the LSTM models, gradient clipping was also applied to control unstable gradient updates, which can occur in recurrent neural networks when learning from sequential financial data.

The validation period was used only for model selection and was not used for final performance reporting. After each epoch, validation loss was calculated using the same portfolio-level objective. This helped compare different parameter settings without selecting a model directly on the test period. Since the validation loss included negative Sharpe ratio, turnover penalty, and drawdown penalty, a lower validation loss indicated a stronger allocation strategy rather than simply better return prediction. Based on this selection process, MLP Experiment 4 and LSTM Experiment 3 were fixed as the final neural network configurations for the next stage of evaluation.

IV. Results and Evaluation

The selected MLP and LSTM models were evaluated on the unseen test period from May, 2024 to December, 2025. The purpose of this evaluation was to compare the traditional MVO benchmark with the two neural network allocation models under the same test-period market conditions. Performance was measured using cumulative return, annual return, annual volatility, Sharpe ratio, and maximum drawdown. Since the final MLP and LSTM models were selected using validation loss, the overall backtest results represent the performance of MLP Experiment 4 and LSTM Experiment 3 rather than the best model chosen directly from the test set.

A. Overall Backtest Performance

Fig. 7 presents the overall test-period performance for the MVO benchmark, selected MLP model, and selected LSTM model in a table. The MLP model achieved the strongest return performance among the three approaches. It generated a cumulative return of 87.76%, compared with 31.86% for MVO and 44.18% for LSTM. The same pattern is observed in annual return, where MLP produced 40.99%, while MVO and LSTM produced 17.72% and 23.67%, respectively. This shows that the MLP model was the most effective approach for return generation during the test period.

However, the higher return of MLP was also associated with higher volatility. The MLP portfolio had annual volatility of 19.46%, compared with 10.52% for MVO and 13.79% for LSTM. This indicates that MLP took larger return fluctuations to generate higher portfolio growth. Despite this higher volatility, MLP still achieved the strongest Sharpe ratio of 2.1061. This is important because the Sharpe ratio adjusts return by risk, so the result shows that MLP did not only increase return in absolute terms but also produced the highest return per unit of volatility. MVO achieved a Sharpe ratio of 1.6842, while LSTM achieved a slightly higher Sharpe ratio of 1.7167.

The drawdown results provide a different view of risk. MVO had the lowest maximum drawdown at -9.17%, followed by MLP at -10.16% and LSTM at -14.69%. This means that MVO remained the most conservative model in terms of peak-to-trough loss during the test period. MLP had slightly higher drawdown than MVO, but the difference was small relative to the improvement in cumulative return and Sharpe ratio. LSTM improved return and Sharpe ratio relative to MVO, but it also had the largest maximum drawdown among the selected models. Therefore, the overall backtest suggests that MLP provided the strongest balance between return generation and risk-adjusted performance, while MVO remained the most defensive portfolio.

Metric	MVO	MLP	LSTM
Period	2024-05-17 → 2025-12-30		
Cumulative Return	31.86%	87.76%	44.18%
Annual Return	17.72%	40.99%	23.67%
Annual Volatility	10.52%	19.46%	13.79%
Sharpe Ratio	1.6842	2.1061	1.7167
Max Drawdown	-9.17%	-10.16%	-14.69%

Fig. 7. Backtest Performance Summary

The cumulative return plot in Fig. 8 supports the same interpretation. During the early part of the test period, the three strategies moved relatively close to each other. However, MLP began to separate more clearly after the first quarter of 2025 and continued to compound at a faster rate through the remainder of the test period. By the end of the backtest, MLP reached the highest portfolio value, while LSTM finished above MVO but remained far below MLP. This visual pattern confirms that the MLP result was not driven only by a small end-period movement; rather, the model showed a stronger upward trend across a substantial part of the test window.

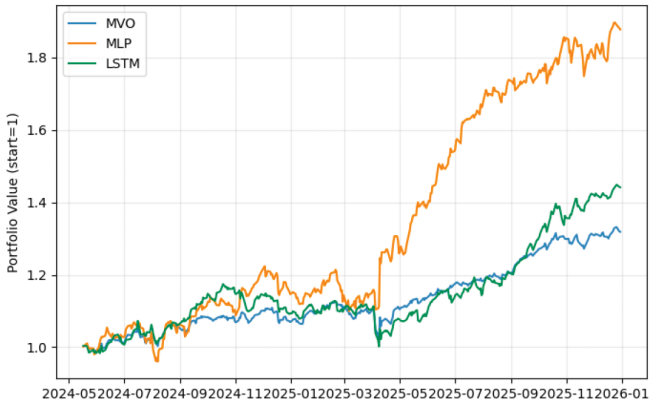

Fig. 8. Cumulative Returns for MVO vs MLP vs LSTM

The rolling Sharpe ratio (63 days) plot in Fig. 9 provides additional insight into the stability of risk-adjusted performance over time. The rolling Sharpe ratio varied significantly across all three models, which is expected in financial time series data because short-term market conditions change over time. MLP showed a strong increase in rolling Sharpe around mid-2025, reaching a much higher peak than the other models. This period also corresponds to the stage where the cumulative return curve for MLP began rising sharply. However, the rolling Sharpe for MLP declined later in the test period, which shows that its risk-adjusted advantage was not equally stable across the full period. LSTM showed periods of strong rolling Sharpe later in 2025, but its final selected-model drawdown remained higher than the other two models. MVO had a more moderate rolling Sharpe pattern and remained closer to the conservative benchmark behavior.

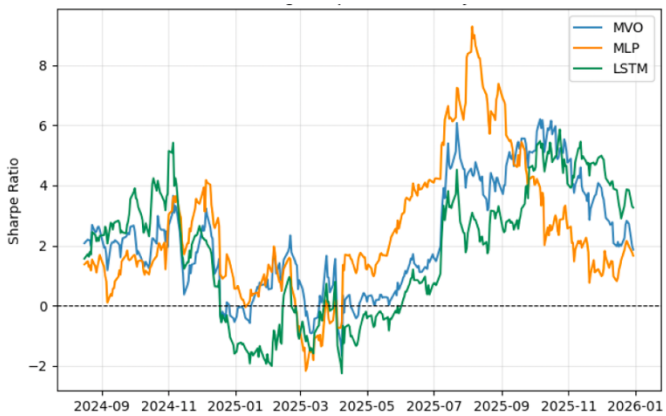

Fig. 9. Rolling Sharpe Ratio (63 day)

The risk-return profile in Fig. 10 summarizes the trade-off between annualized return and annualized volatility. MVO appears in the lower-risk and lower-return region, with 10.52% annual volatility and 17.72% annual return. LSTM moved upward and to the right, meaning it generated higher return than MVO but also took higher volatility. MLP moved further upward and to the right, producing the highest return and highest volatility. The Sharpe reference lines show that MLP lies above the Sharpe 2.0 line, while MVO and LSTM lie closer to the lower Sharpe bands. This supports the backtest table because MLP had the best risk-adjusted performance even though it was also the most volatile strategy.

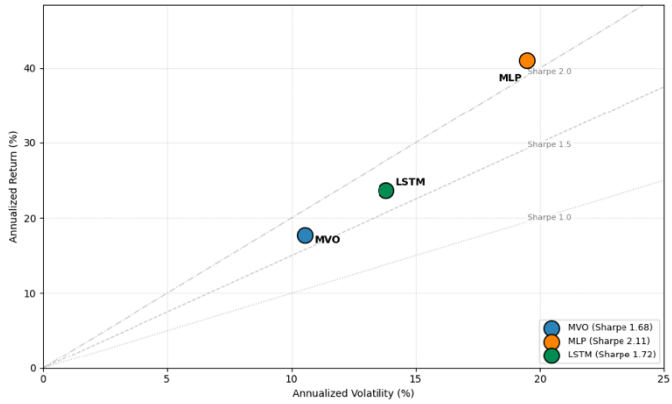

Fig. 10. Risk-Return Profile: MVO vs MLP vs LSTM

The asset allocation comparison in Fig. 11 explains part of the difference in portfolio behavior. MVO was constrained by the 20% asset cap and allocated 20% each to GLD, QQQ, SPY, and TLT, with additional allocation to LQD and XLE. In contrast, the selected MLP model placed much larger average weight on QQQ at around 42%, followed by TLT and XLE at around 22% each. This shows that the MLP model was more concentrated in assets that contributed strongly to test-period growth, especially QQQ. The LSTM model was more diversified, with its largest average allocation going to GLD at around 28%, followed by LQD and IWM at around 17% each, and smaller allocations across several other ETFs. This broader allocation helps explain why LSTM had more moderate return than MLP, but it does not fully explain its higher drawdown during the selected test period. Overall, the allocation chart shows that the neural networks learned meaningfully different allocation patterns rather than simply replicating the MVO benchmark.

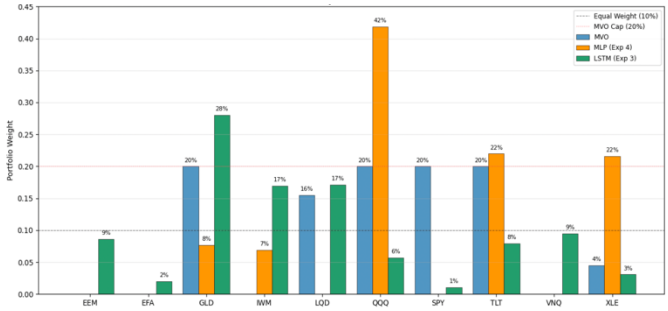

Fig. 11. Asset Allocation Comparison: MVO vs MLP vs LSTM

B. *Experiment-Level Comparison*

Fig. 12 presents the experiment-level comparison across all MLP and LSTM configurations in a table. This comparison was included to understand how different parameter settings affected validation loss, test Sharpe ratio, and maximum drawdown. Since the final model selection was based on validation loss, MLP Experiment 4 and LSTM Experiment 3 were used in the main backtest comparison. However, the experiment-level results also show how alternative configurations performed under test-period conditions.

For the MLP experiments, Experiment 4 was the strongest configuration, achieving the lowest validation loss of -1.7935, the highest test Sharpe ratio of 2.1061, and the lowest drawdown among the MLP configurations at -10.16%. In contrast, the LSTM results were more mixed. Experiment 3 had the lowest validation loss of -1.3259 and was used in the main backtest, but Experiment 2 performed better on test-period Sharpe ratio and drawdown, with a Sharpe ratio of 1.9631 and maximum drawdown of -10.40%, compared with 1.7167 and -14.69% for Experiment 3. This suggests that LSTM Experiment 2 can be considered an alternative configuration when the focus is test-period risk-adjusted performance.

Therefore, the experiment-level comparison provides two important observations. First, MLP Experiment 4 was consistently the strongest MLP configuration and also the strongest overall model in the experiment table. Second, LSTM Experiment 2 should be considered as an alternative LSTM configuration for further performance evaluation because it delivered better test-period Sharpe ratio and drawdown than the validation-selected LSTM model.

Model	Label	Best Val Loss	Test Sharpe	Max Drawdown
MLP	Baseline	-1.3535	1.5417	-22.82%
MLP	Exp 1	-1.7017	1.2898	-15.06%
MLP	Exp 2	-1.6853	1.4264	-17.29%
MLP	Exp 3	-1.3362	1.8059	-16.67%
MLP	Exp 4	-1.7935	2.1061	-10.16%
LSTM	Baseline	-1.3128	1.3781	-12.21%
LSTM	Exp 1	-1.2131	1.7845	-13.57%
LSTM	Exp 2	-1.2270	1.9631	-10.40%
LSTM	Exp 3	-1.3259	1.7167	-14.69%
LSTM	Exp 4	-1.1745	1.5778	-11.20%

Fig. 12. Experiment Comparison across all Configurations

The hyperparameter sensitivity heatmap in Fig. 13 provides a visual summary of these experiment-level results. It shows that MLP Experiment 4 had the strongest combined outcome across validation loss, Sharpe ratio, and drawdown. This supports the earlier observation that the MLP model was more consistent across validation and test-period performance. For the LSTM models, the heatmap shows a clearer difference between validation-based ranking and test-metric ranking. LSTM Experiment 3 performed best on validation loss, while LSTM Experiment 2 performed better on test Sharpe ratio and drawdown. This indicates that the LSTM results were more sensitive to parameter choices and evaluation criteria.

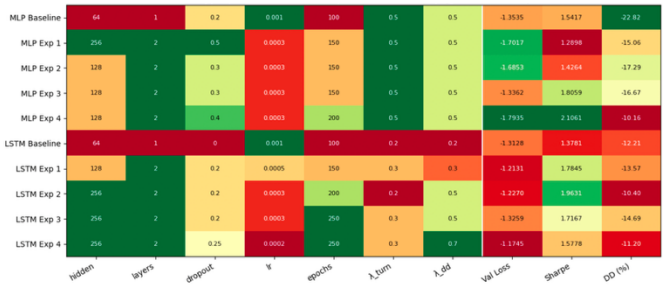

Fig. 13. Hyperparameter Sensitivity & Metric Heatmap (green = better)

V. Conclusion

Overall, the results show that direct portfolio allocation through machine learning improved performance relative to the MVO benchmark during the test period. The selected MLP model produced the highest cumulative return, annual return, and Sharpe ratio while keeping drawdown close to the benchmark. The selected LSTM model also improved annual return and Sharpe ratio relative to MVO, but it experienced a larger drawdown. At the experiment level, MLP Experiment 4 remained the strongest configuration, while LSTM Experiment 2 appeared as a useful alternative for further evaluation. Therefore, the main result is that end-to-end learning can improve portfolio allocation performance, but the strength of the result depends on both model architecture and parameter selection.

Moreover, any future work can improve this project by using walk-forward validation instead of a single train-validation-test split, so the models can be tested across multiple market regimes. The backtest can also be made more realistic by adding transaction costs, rebalancing frequency, and additional benchmarks such as equal-weight, risk parity, and minimum-variance portfolios. Finally, future versions can include macroeconomic indicators, volatility indexes, and more advanced architectures such as GRU or Transformer models to improve stability and interpretability

References

[1] H. Markowitz, “Portfolio Selection,” The Journal of Finance, vol. 7, no. 1, pp. 77–91, Mar. 1952.

[2] S. Gu, B. Kelly, and D. Xiu, “Empirical Asset Pricing via Machine Learning,” The Review of Financial Studies, vol. 33, no. 5, pp. 2223–2273, May 2020.

[3] J. B. Heaton, N. G. Polson, and J. H. Witte, “Deep learning for finance: deep portfolios,” Applied Stochastic Models in Business and Industry, vol. 33, no. 1, pp. 3–12, Jan. 2017.

[4] Z. Zhang, S. Zohren, and S. Roberts, “Deep Learning for Portfolio Optimisation,” The Journal of Financial Data Science, vol. 2, no. 4, pp. 8–20, Fall 2020.

[5] R. Aroussi, “yfinance: Download market data from Yahoo! Finance’s API,” GitHub, 2026. [Online]. Available: https://github.com/ranaroussi/yfinance

[6] W. F. Sharpe, “Mutual Fund Performance,” The Journal of Business, vol. 39, no. 1, pp. 119–138, Jan. 1966.

[7] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” Neural Computation, vol. 9, no. 8, pp. 1735–1780, Nov. 1997.